

Estudo de Caso 1

Teste e Qualidade de Software

MediClinic — Do requisito ao release

O desafio

Vocês acabaram de entrar para a equipe de QA da **MediClinic**, uma empresa que está desenvolvendo uma plataforma para agendamento de consultas médicas.

A primeira versão do sistema está sendo desenvolvida por uma equipe pequena e deverá ser disponibilizada em breve.

A Product Owner, Ana, marcou uma reunião com a equipe.

Ana — Product Owner:

“Precisamos colocar essa primeira versão no ar em breve. O desenvolvimento já começou, mas eu não quero que QA apareça apenas no final para descobrir que alguma coisa está errada. Quero que vocês acompanhem o projeto desde agora.”

Vocês serão responsáveis por acompanhar a qualidade da aplicação durante a sprint.

Ao longo do projeto, novas situações aparecerão e vocês precisarão tomar decisões como uma equipe de QA.

O objetivo não é apenas executar testes. Vocês deverão demonstrar **como a qualidade pode ser construída, avaliada e acompanhada durante todo o ciclo de desenvolvimento**.

Importante: este projeto não aborda testes de performance.

ETAPA 1 — A primeira reunião

Ana apresenta os requisitos definidos para a primeira versão da MediClinic.

Requisitos funcionais

1. RF01 — Cadastro de paciente

O paciente deve conseguir criar uma conta informando:

- nome;
- e-mail;
- CPF;
- senha.

O e-mail deve ser único no sistema.

2. RF02 — Login

O paciente deve conseguir realizar login utilizando e-mail e senha.

Operações protegidas devem exigir autenticação.

3. RF03 — Médicos

O sistema deve disponibilizar uma lista de médicos contendo:

- nome;
- CRM;
- especialidade;
- status.

Somente médicos ativos podem receber novos agendamentos.

4. RF04 — Disponibilidade

O paciente deve conseguir consultar os horários disponíveis de um médico.

Um horário já ocupado não deve aparecer como disponível.

5. RF05 — Agendamento

O paciente deve conseguir selecionar um médico, uma data e um horário para realizar uma consulta.

Após o agendamento, o sistema deve gerar um identificador único para a consulta.

6. RF06 — Antecedência mínima

Uma consulta somente poderá ser agendada se estiver a pelo menos **24 horas** do momento atual.

Consultas com menos de 24 horas de antecedência devem ser rejeitadas.

7. RF07 — Período máximo

Uma consulta não poderá ser agendada para mais de **90 dias** no futuro.

8. RF08 — Conflito de horários

Um paciente não pode possuir duas consultas no mesmo horário.

Dois pacientes também não podem agendar o mesmo médico no mesmo horário.

9. RF09 — Visualização de consultas

O paciente deve conseguir visualizar suas próprias consultas.

Um paciente não deve conseguir visualizar as consultas de outro paciente.

10. RF10 — Cancelamento pelo paciente

O paciente poderá cancelar sua própria consulta desde que ela esteja a pelo menos 24 horas de distância.

11. RF11 — Cancelamento pelo administrador

Um administrador poderá cancelar qualquer consulta futura, independentemente da regra de 24 horas.

12. RF12 — Consulta passada

Uma consulta que já ocorreu não poderá ser cancelada.

13. RF13 — Confirmação do agendamento

Após um agendamento realizado com sucesso, o sistema deve apresentar:

- identificador da consulta;
- médico;
- especialidade;
- data;
- horário.

RF14 — Dados corretos

O paciente deve informar seus dados corretamente durante o cadastro.

Ana termina a apresentação.

Ana:

“Esses são os requisitos. Podem começar a preparar os testes.”

Vocês, porém, percebem imediatamente um problema.

O requisito **RF14** diz que o paciente deve informar seus dados “corretamente”.

Mas o que significa “corretamente”?

O requisito não informa, por exemplo:

- qual formato o CPF deve possuir;
- quais são as regras para o e-mail;
- se existe tamanho mínimo ou máximo para o nome;
- quais são as regras da senha;
- o que acontece quando um dado é inválido.

O problema

Se vocês não sabem exatamente qual comportamento é esperado, como poderão saber se o sistema está correto?

Desafio da equipe de QA

Antes de pensar em casos de teste:

1. analisem o RF14;
 2. identifiquem ambiguidades;
 3. identifiquem informações faltantes;
 4. expliquem por que o requisito não é suficientemente testável;
 5. proponham uma versão mais clara e verificável do requisito.
-

ETAPA 2 — “Mas onde está o maior risco?”

Após a reunião, vocês conversam com Marcos, o Tech Lead.

Marcos — Tech Lead:

“Concordo que precisamos melhorar os requisitos. Mas temos outra questão: não teremos tempo para testar tudo da mesma maneira. Precisamos descobrir o que pode causar mais problemas para o negócio.”

A equipe percebe que não basta perguntar:

“O que podemos testar?”

É preciso perguntar:

“O que pode dar errado e qual seria o impacto?”

A partir dos requisitos da MediClinic, identifiquem pelo menos **5 riscos**.

Para cada risco, determinem:

- probabilidade;
- impacto;
- prioridade;
- estratégia de teste.

Por exemplo:

Dois pacientes conseguem agendar o mesmo médico no mesmo horário.

Esse problema poderia causar conflito operacional e prejudicar diretamente os pacientes.

Registrem os riscos identificados e justifiquem as classificações de prioridade.

ETAPA 3 — “Como vamos garantir que não esquecemos nada?”

Na reunião seguinte, Marcos faz uma nova pergunta:

Marcos:

“Vocês identificaram os riscos. Mas, quando terminarmos o projeto, como vamos provar que os requisitos importantes realmente foram validados?”

A equipe decide utilizar uma **matriz de rastreabilidade**.

Mas existe uma questão importante:

Neste momento, vocês ainda **não possuem casos de teste**.

Portanto, não devem inventar uma coluna de casos de teste que ainda não existe.

A primeira versão da matriz deverá representar aquilo que a equipe já conhece:

Requisito → Necessidade/Cenário → Risco → Prioridade

Por exemplo:

Requisito	Necessidade	Risco	Prioridade
RF05	Realizar agendamento	Consulta não ser criada corretamente	Alta
RF06	Respeitar antecedência mínima	Sistema aceitar agendamento inválido	Crítica
RF08	Evitar conflitos	Dois pacientes ocuparem o mesmo horário	Crítica

Criem a matriz inicial para os requisitos analisados.

A matriz não é um documento que será criado uma única vez.

Ela será atualizada conforme novos artefatos forem produzidos durante a sprint.

ETAPA 4 — “Agora sabemos os riscos. Como vamos testar?”

No dia seguinte, Rafael, o desenvolvedor, envia uma mensagem:

Rafael — Desenvolvedor:

“Estou começando a implementar a parte de agendamento. Vocês conseguem me dizer quais situações precisam ser consideradas?”

A equipe percebe que dizer apenas:

“Teste o agendamento.”

não é suficiente.

Vocês precisam determinar **quais entradas e situações devem ser consideradas**.

A equipe começa pela regra que foi classificada como um dos maiores riscos:

Uma consulta deve ser agendada com pelo menos **24 horas** e no máximo **90 dias** de antecedência.

O primeiro problema: quais são as classes de entrada?

Utilizem **particionamento de equivalência** para dividir as possíveis entradas em classes.

Considerem:

- menos de 24 horas;
- exatamente 24 horas;
- entre 24 horas e 90 dias;
- exatamente 90 dias;
- mais de 90 dias.

Identifiquem:

- classes válidas;
 - classes inválidas;
 - pelo menos um representante de cada classe.
-

O segundo problema: e se o erro estiver na fronteira?

Marcos interrompe a discussão:

Marcos:

“Vocês estão testando 20 horas e 50 dias. Mas e se o erro estiver justamente na fronteira?”

A equipe decide utilizar **análise de valor limite**.

Considerem situações como:

- 23h59;
- exatamente 24h;
- 24h01;
- 89 dias 23h59;
- exatamente 90 dias;
- 90 dias + 1 minuto.

Expliquem por que esses valores são relevantes.

ETAPA 5 — “O cancelamento está mais complicado do que parecia”

Enquanto vocês trabalham no agendamento, Ana envia uma nova mensagem:

Ana:

“Precisamos definir os testes de cancelamento também. Temos algumas regras diferentes dependendo de quem está tentando cancelar.”

Ela relembra as regras:

- o paciente precisa estar autenticado;
- o paciente só pode cancelar sua própria consulta;
- uma consulta passada não pode ser cancelada;
- o paciente precisa ter pelo menos 24 horas de antecedência;
- o administrador pode cancelar uma consulta futura independentemente das 24 horas.

Marcos comenta:

Marcos:

“Temos várias condições combinadas. Se simplesmente criarmos casos aleatoriamente, podemos esquecer alguma combinação importante.”

A equipe decide utilizar uma **tabela de decisão**.

Criem uma tabela contendo:

- condições;
- combinações relevantes;
- ações esperadas.

A partir dela, identifiquem os cenários que posteriormente deverão ser transformados em casos de teste.

ETAPA 6 — “Uma consulta não está sempre no mesmo estado”

Durante a discussão, Rafael pergunta:

Rafael:

“E o que acontece com uma consulta depois que ela é criada? Podemos cancelá-la depois que já foi realizada?”

A equipe percebe que algumas regras dependem do **estado atual da consulta**.

Uma consulta pode, por exemplo, estar:

Disponível → Agendada → Realizada

ou

Disponível → Agendada → Cancelada

A equipe decide representar esse comportamento utilizando uma **máquina de estados**.

Modelem o ciclo de vida da consulta.

Identifiquem:

- estados;
- eventos;
- transições válidas;
- transições inválidas.

Depois, derivem cenários de teste a partir dessas transições.

ETAPA 7 — “Agora transformem tudo isso em casos de teste”

Depois das atividades anteriores, vocês já possuem:

- requisitos analisados;
- riscos;
- classes de equivalência;
- valores limite;
- combinações da tabela de decisão;
- estados e transições.

Marcos envia uma nova mensagem:

Marcos:

“Agora vocês já sabem o que precisa ser validado. Transformem esses cenários em casos de teste que possam ser executados por qualquer pessoa da equipe.”

Criem pelo menos **15 casos de teste**.

Cada caso deverá conter:

Campo	Descrição
ID	Identificador único
Requisito	Requisito relacionado
Título	O que será validado
Prioridade	Baixa, Média, Alta ou Crítica
Pré-condições	Estado necessário
Dados	Dados utilizados
Passos	Procedimento
Resultado esperado	Comportamento esperado
Técnica	Técnica utilizada

Os casos deverão utilizar diferentes técnicas de teste estudadas.

ETAPA 8 — “Voltamos para a matriz”

Agora existe algo que não existia no início do projeto:

casos de teste.

Marcos pede:

Marcos:

“Atualizem a matriz. Quero conseguir sair de um requisito e chegar até o teste que o valida.”

A matriz passa a evoluir.

Agora vocês podem relacionar:

Requisito → Risco → Técnica → Cenário → Caso de teste

Por exemplo:

Requisito	Risco	Técnica	Cenário	Caso
RF06	Agendamento inválido aceito	Valor limite	Exatamente 24h	CT-006
RF08	Conflito de horário	Equivalência	Horário ocupado	CT-008
RF10	Cancelamento indevido	Tabela de decisão	Menos de 24h	CT-010

Atualizem a matriz com os casos criados.

ETAPA 9 — “Precisamos falar a mesma língua”

Durante uma reunião com Ana e Rafael, surge uma nova dificuldade.

Ana explica:

Ana:

“Eu consigo entender os casos de teste, mas gostaria que os comportamentos mais importantes do sistema fossem descritos de uma maneira que negócio, desenvolvimento e QA consigam entender.”

Marcos sugere utilizar **BDD**.

A equipe escolhe alguns dos comportamentos mais importantes já identificados.

Agora vocês deverão transformar esses comportamentos em cenários utilizando **Gherkin**.

Criem pelo menos **3 cenários**, sendo:

- pelo menos um cenário de sucesso;
- pelo menos um cenário de falha/rejeição.

Exemplo:

Feature: Agendamento de consultas

Scenario: Agendar uma consulta válida

Given que o paciente está autenticado

And existe um médico ativo

And existe um horário disponível

When o paciente agenda a consulta

Then a consulta deve ser criada com sucesso

And o sistema deve retornar o identificador da consulta

Os cenários BDD devem representar comportamentos já identificados durante o projeto.

ETAPA 10 — “A matriz precisa acompanhar isso também”

Agora vocês possuem outro artefato:

cenários BDD.

Marcos pede uma nova atualização da matriz.

Ela passa a relacionar:

Requisito → Caso de teste → Cenário BDD

Por exemplo:

Requisito	Caso	Cenário BDD
RF05	CT-005	Agendar consulta válida
RF06	CT-006	Rejeitar consulta com menos de 24h
RF10	CT-010	Cancelar consulta dentro das regras

Atualizem a matriz.

ETAPA 11 — “Vamos testar antes de terminar o código”

Rafael informa:

Rafael:

“Vou implementar agora a regra de antecedência. Se vocês já souberem o comportamento esperado, podemos trabalhar juntos.”

Marcos responde:

Marcos:

“Então vamos fazer diferente. Vamos criar os testes antes da implementação.”

A equipe decide aplicar **TDD — Test-Driven Development**.

A regra é:

- menos de 24 horas → não pode agendar;
- entre 24 horas e 90 dias → pode agendar;
- mais de 90 dias → não pode agendar.

Para simplificar, Rafael propõe criar uma função Python:

```
def pode_agendar(horas_antecedencia):  
    ...
```

A equipe deverá trabalhar utilizando o ciclo:

Red

Criar um teste que represente o comportamento esperado.

Executar o teste e observar a falha.

Green

Implementar o mínimo necessário para fazer o teste passar.

Refactor

Melhorar a implementação mantendo o comportamento esperado.

Documentem o que aconteceu em cada etapa.

ETAPA 12 — “Temos muitos cenários parecidos”

Depois do primeiro ciclo de TDD, Rafael percebe:

Rafael:

“Estamos escrevendo vários testes muito parecidos. Não seria melhor organizar esses cenários?”

A equipe decide utilizar **pytest** para automatizar os testes.

Criem testes automatizados para:

- menos de 24h;
- exatamente 24h;
- entre 24h e 90 dias;
- exatamente 90 dias;
- mais de 90 dias.

Utilizem **parametrização** quando fizer sentido.

Os testes devem representar os cenários que foram definidos anteriormente por meio das técnicas de teste.

ETAPA 13 — “Todos passaram. Mas quanto realmente testamos?”

Depois de executar os testes, todos estão passando.

Rafael comenta:

Rafael:

“Então está tudo testado.”

Marcos não concorda:

Marcos:

“Todos os testes passarem significa que testamos tudo?”

A equipe decide analisar a **cobertura dos testes** utilizando o ambiente e as ferramentas de testes em Python que já conhece.

Avaliem a cobertura obtida pelos testes automatizados.

Identifiquem:

- quais partes do código foram exercitadas;
- quais partes não foram;
- se existem cenários importantes que não estão representados.

Depois discutam:

É possível ter 100% de cobertura de código e ainda existir um defeito?

Justifiquem a resposta.

ETAPA 14 — “Recebemos os resultados da execução”

Agora a equipe recebe o relatório da execução dos testes.

O resultado foi:

5 testes executados

4 aprovados

1 reprovado

O teste que falhou corresponde ao seguinte cenário:

CT-007 — Agendamento exatamente 24 horas antes

Entrada: consulta exatamente 24 horas no futuro.

Resultado esperado: o agendamento deve ser aceito.

Resultado obtido: o agendamento foi rejeitado.

Rafael informa:

Rafael:

“Recebi o resultado. Vou verificar esse problema.”

Nesse momento, a equipe não precisa investigar a implementação do código.

O objetivo é trabalhar a partir da **evidência de teste**.

Registrem o defeito encontrado.

O relatório deverá conter:

- ID;
 - título;
 - requisito relacionado;
 - caso de teste relacionado;
 - descrição;
 - resultado esperado;
 - resultado observado;
 - severidade;
 - prioridade.
-

ETAPA 15 — “Não encontramos apenas um problema”

No decorrer da sprint, a equipe recebe mais duas ocorrências identificadas durante a validação do sistema.

1. BUG-002 — Acesso à consulta de outro paciente

Situação observada

Um paciente autenticado conseguiu visualizar uma consulta pertencente a outro paciente.

Resultado esperado

Um paciente deve conseguir visualizar somente suas próprias consultas.

Requisito relacionado

RF09.

2. BUG-003 — Agendamento com médico inativo

Situação observada

O sistema permitiu que um paciente agendasse uma consulta com um médico cujo status estava definido como **Inativo**.

Resultado esperado

Somente médicos ativos podem receber novos agendamentos.

Requisito relacionado

RF03.

Agora a equipe possui três defeitos:

ID	Problema	Requisito
BUG-001	Exatamente 24h é rejeitado	RF06
BUG-002	Paciente acessa consulta de outro paciente	RF09
BUG-003	Agendamento com médico inativo	RF03

Para cada um, registrem o defeito de forma estruturada.

ETAPA 16 — “Qual problema devemos corrigir primeiro?”

Ana pergunta:

Ana:

“Temos três defeitos. Qual deles é mais grave? E qual precisa ser corrigido primeiro?”

Agora vocês precisam diferenciar:

Severidade

Qual é o impacto do defeito?

Prioridade

Qual é a urgência de correção?

Classifiquem os três defeitos.

Depois, escolham pelo menos dois deles e expliquem por que **severidade e prioridade podem ser diferentes**.

ETAPA 17 — “Os desenvolvedores corrigiram os problemas”

Rafael envia uma nova mensagem:

Rafael:

“Os três problemas reportados foram corrigidos. A equipe de desenvolvimento liberou uma nova versão para validação.”

Agora a equipe precisa decidir como verificar as correções.

Para cada defeito, identifiquem o **caso de teste que deve ser repetido para confirmar a correção**.

Isso é o **reteste**.

Por exemplo:

BUG-001 está relacionado ao CT-007.

Portanto, o CT-007 deve ser executado novamente.

Façam essa associação para os três defeitos.

ETAPA 18 — “E o que mais precisamos testar?”

Marcos faz uma nova pergunta:

Marcos:

“Se o desenvolvedor alterou o código para corrigir esses problemas, basta executar os três testes que falharam?”

A equipe percebe que não necessariamente.

Uma alteração pode corrigir um comportamento e afetar outro.

Agora vocês deverão planejar o **teste de regressão**.

Não será necessário executar os testes.

O objetivo é decidir **quais testes deveriam ser executados novamente** após cada correção.

Por exemplo:

BUG-001

Relacionado à regra de antecedência.

Além do teste exatamente 24h, considerem os demais testes relacionados aos limites:

- 23h59;
- 24h;
- 24h01;
- 90 dias;
- acima de 90 dias.

BUG-002

Relacionado ao acesso às consultas.

Considerem também outros casos relacionados a:

- paciente autenticado;
- visualização da própria consulta;
- tentativa de acesso indevido;
- autenticação.

BUG-003

Relacionado ao cadastro/status de médicos.

Considerem também:

- médico ativo;
- médico inativo;
- disponibilidade;
- agendamento de médico ativo;
- tentativa de agendamento de médico inativo.

Para cada defeito, definam:

1. qual teste deve ser utilizado no **reteste**;
2. quais testes devem fazer parte da **regressão**;
3. por que esses testes foram escolhidos.

ETAPA 19 — “Recebemos os resultados”

Agora a equipe recebe os resultados fornecidos pelo ambiente de validação.

BUG-001

Reteste: aprovado.

O agendamento exatamente 24 horas antes agora é aceito.

Regressão: aprovada.

Os demais testes da regra de antecedência continuam funcionando.

BUG-002

Reteste: aprovado.

Um paciente não consegue mais visualizar consultas pertencentes a outro paciente.

Regressão: aprovada.

Os testes relacionados à visualização das consultas continuam funcionando.

BUG-003

Reteste: aprovado.

Um médico inativo não pode mais receber novos agendamentos.

Regressão: aprovada.

Os testes relacionados a médicos ativos e disponibilidade continuam funcionando.

A equipe agora possui evidências suficientes para atualizar seus registros.

ETAPA 20 — “Atualizem a matriz uma última vez”

Marcos escolhe um requisito crítico e pergunta:

Marcos:

“Quero saber tudo o que fizemos em relação a esse requisito, desde o momento em que identificamos o risco até o resultado final.”

Agora a matriz deverá ser atualizada novamente.

Ela pode conectar:

Requisito → Risco → Técnica → Caso de teste → BDD → pytest → Defeito → Reteste

Por exemplo:

Requisito	Risco	Técnica	Caso	BDD	pytest	Defeito	Reteste
RF06	Agendamento inválido aceito	Valor limite	CT-007	Scenario 02	test_24h	BUG-001	Aprovado

Completem a matriz final utilizando os artefatos produzidos durante o projeto.

ETAPA 21 — “Podemos liberar?”

Chegou o dia do release.

Ana chama a equipe para a última reunião.

Ana:

“Precisamos tomar uma decisão. Podemos colocar a MediClinic em produção?”

Vocês agora possuem evidências de todo o trabalho realizado.

A decisão não pode ser baseada simplesmente em:

“A maioria dos testes passou.”

A equipe deverá analisar:

Testes

- quantidade executada;
- quantidade aprovada;
- quantidade reprovada;
- testes bloqueados.

Automação

- quais cenários foram automatizados;
- cobertura obtida;
- limitações da cobertura.

Defeitos

- quantidade encontrada;
- quantidade corrigida;
- severidade;
- prioridade;
- resultados do reteste.

Regressão

- quais testes foram selecionados;
- por que foram selecionados;
- resultado da regressão.

Riscos

- quais riscos foram mitigados;

- quais permanecem;
 - quais podem impedir o release.
-

ETAPA 22 — A decisão de release

A equipe deverá recomendar uma das três opções:

GO

A versão pode ser liberada.

NO-GO

A versão não deve ser liberada.

CONDITIONAL GO

A versão pode ser liberada, desde que determinadas condições sejam atendidas.

A recomendação deve ser fundamentada nas evidências produzidas durante o projeto.

ETAPA 23 — “O que aprendemos sobre qualidade?”

Depois da decisão de release, Marcos encerra a sprint com uma retrospectiva.

Marcos:

“Quando vocês chegaram aqui, talvez pensassem que o trabalho de QA começava quando o código estava pronto. Depois dessa sprint, vocês ainda pensam da mesma forma?”

Com base na experiência do projeto, discutam:

1. Em que momento a equipe começou a contribuir para a qualidade?
 2. Qual problema poderia ter sido identificado antes de o código existir?
 3. Por que a análise de riscos foi importante para decidir onde concentrar os testes?
 4. Como o particionamento de equivalência e a análise de valor limite ajudaram a construir os testes?
 5. Por que a tabela de decisão foi adequada para o problema do cancelamento?
 6. O que a máquina de estados permitiu identificar?
 7. Qual é a diferença entre um caso de teste e um cenário BDD?
 8. Qual é a diferença entre BDD e TDD?
 9. Por que TDD pode ajudar a identificar problemas antes da implementação estar completa?
 10. Qual é a diferença entre cobertura de código e cobertura de requisitos?
 11. Por que 100% de cobertura não significa ausência de defeitos?
 12. Qual é a diferença entre reteste e regressão?
 13. Por que a matriz de rastreabilidade precisou ser atualizada durante o projeto?
 15. Ao final do projeto, o que significa, na prática, dizer que “o software possui qualidade”?
-

ENTREGA FINAL

A equipe deverá entregar um único **Dossiê de QA da MediClinic**, reunindo os artefatos produzidos durante a sprint:

1. análise e refinamento dos requisitos;
2. análise de qualidade;
3. análise de riscos;
4. matriz de rastreabilidade inicial;
5. particionamento de equivalência;
6. análise de valor limite;
7. tabela de decisão;
8. máquina de estados;
9. casos de teste;
10. cenários BDD/Gherkin;
11. evidências do processo de TDD;
12. testes automatizados com pytest;
13. análise de cobertura;

14. registros dos três defeitos;
 15. classificação de severidade e prioridade;
 16. planejamento de reteste;
 17. planejamento de regressão;
 18. matriz de rastreabilidade final;
 19. decisão de release;
 20. retrospectiva.
-